

Project 03 — Offline Field Assistant on llama.cpp

Mobile · fully on-device · no network, no data egress

Contents

Project 03 — Offline Field Assistant (llama.cpp)	1
0. Numbered build plan	1
1. Brief	2
2. Why llama.cpp for this workload	2
3. Model and compression	2
4. Architecture	3
5. Frontend	4
6. Architecture graph: node inventory	4
7. The improvement loop for this project	6
7a. LoRA adapters: where they earn their place here	6
8. The release gate for this project	7
9. Monitoring and KPIs	7
10. Security, audit and compliance	8
11. Deployment	8
12. Deliverables	8
13. Out of scope	8

Project 03 — Offline Field Assistant (llama.cpp)

Read 00-shared-requirements.md first. Everything in it applies here, with the device-class substitutions in §4.

0. Numbered build plan

Paste this section into Claude Code’s plan mode to scope the work. Each step is independently reviewable; steps 1–4 must land before the demo is filmable.

1. **Repository skeleton.** Monorepo with frontend/, backend/, model/, deploy/, observability/. Strict TypeScript, typed Python, lint + types + tests in CI from the first commit.
2. **llama.cpp serving layer.** Stand up the engine with the compression spec in §3–§4 below. Record weights and latency before and after, and state what the freed resource buys in the units the client pays for.
3. **Backend API.** The component chain in §5 as real services: authentication, authorization, adversarial-input detection, context assembly, inference, guardrails, structured logging, hash-chained audit log.
4. **Product tab.** The working demo, usable by someone who has never seen it, plus the configuration surface. Configuration changes must visibly change behaviour and be inspectable before they apply.
5. **Architecture tab.** The 3D expandable-pipeline graph, built to the spec in 00-shared-requirements.md §2. Use the node inventory in §6 of this document as the content. Click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel with all four rationale fields plus a “Say this” line.
6. **Guided tour.** An ordered walk covering every node that carries the commercial argument, including the honest stop about what is *not* enabled. Arrow keys, autoplay, and it must run with the backend stopped.

7. **Monitoring tab.** Quality, Performance, Security, Audit, Improvement. Live data when available, seeded demo data otherwise, honest source badge — except Improvement, which is never seeded.
8. **Feedback capture.** Rating, side-by-side comparison and correction in the Product tab, per §7. Redact on write, stamp the model/config version, store the triple denormalised, mark what an export consumed.
9. **Export and training path.** JSON Lines in the trainer’s own format, resumable. Training is operator-run, never automatic. Write a training manifest next to every artifact. Evaluate **LoRA adapters** per §7a — that section says where they belong in *this* project and where they do not.
10. **Release gate.** Both axes per §8 — quality and performance, each able to fail the build on its own. Wire it into CI.
11. **Deployment.** Containers, infrastructure as code, staging → production with a rollback, and autoscaling on the signal that actually saturates.
12. **README.** A stranger can reach a working demo. Ends with the cost/performance/accuracy tradeoff table, one row per real decision, and a portability section.

Acceptance is §12 of this document plus the shared deliverables checklist.

1. Brief

Build an **on-device assistant for field workers who have no reliable network**: technicians in plant rooms and basements, maintenance crews on remote sites, clinicians in facilities that forbid cloud upload, inspectors in the field.

It answers questions from a large body of technical documentation — service manuals, safety procedures, part specifications — with everything running locally: the model, the retrieval index, and the audit log.

The defining property:

The device runs in airplane mode for the entire demo. No request leaves it.

That is the product. Not “privacy-conscious” or “encrypted in transit” — the data physically cannot leave, because there is nowhere for it to go. For a client in a regulated industry or a security-cleared site, that is a different category of argument from any cloud assurance.

Target user: industrial services, utilities, defence, healthcare facilities management — organisations whose people work where connectivity is absent and whose data cannot go to a third party regardless.

2. Why llama.cpp for this workload

- **It runs where nothing else does.** llama.cpp is C++ with minimal dependencies and mature backends for Metal (iOS/macOS), Vulkan and NNAPI (Android), and CPU everywhere. No Python runtime, no CUDA, no container.
- **GGUF quantization is the enabling technology, not an optimisation.** A 7B model at Q4_K_M is roughly 4.4 GB against ~14 GB at FP16. On a phone this is not a cost saving — it is the entire difference between the product existing and not existing. That framing belongs in the architecture graph.
- **mmap model loading** means the OS pages weights in on demand. Startup is fast and the app does not need to hold the whole model resident, which matters enormously under mobile memory pressure where the OS kills large processes.
- **It does have a KV cache**, and on-device it is a visible constraint rather than a background detail: context length is bounded by RAM you can actually feel. This makes it an unusually good thing to *demonstrate* — show context filling up and the memory cost rising in real time.

The honest limitation, state it plainly: a 7B model quantized to 4 bits is meaningfully weaker than a frontier model. The system must be designed around that — strong retrieval, narrow scope, and a low threshold for saying “I don’t know”. A project that pretends otherwise will fail its first real user.

3. Model and compression

Component	Choice
LLM	Qwen2.5-3B-Instruct or Llama-3.2-3B-Instruct
Format	GGUF, Q4_K_M primary; ship Q5_K_M and Q8_0 for comparison
Embeddings	bge-small-en-v1.5 GGUF, also on-device
Vector store	SQLite with sqlite-vec, or a flat index — the corpus is bounded

Start at 3B, not 7B. On a mid-range phone a 7B model at Q4 leaves too little headroom for the OS, and the app gets killed in the background. Ship 3B as default and offer 7B as an opt-in for high-end devices. Measure both.

Compression spec - Produce Q4_K_M, Q5_K_M and Q8_0 variants. Record for each: file size, RAM at runtime, tokens/sec on target hardware, and quality-gate score. - **Put that table in the README and in the architecture graph node.** The size/speed/quality trade is the most concrete engineering decision in the project, and showing the measured curve is far more persuasive than asserting a choice. - Consider an importance matrix (imatrx) computed on domain documentation to improve low-bit quality — same principle as calibration data elsewhere in this pack.

Quality gate - Domain suite of held-out questions with reference answers from the documentation, scored on: does it answer from the retrieved passage, does it cite, does it decline when the answer is absent. - Run the gate **on the target device**, not on a workstation. Quantized kernels differ per backend, and Metal, Vulkan and CPU do not produce identical output. A gate that only ran on a laptop has not tested what ships. - Track tokens/sec and time-to-first-token per device tier as gated metrics — correctness that arrives too slowly is still a failure on mobile.

4. Architecture

Same chain as 00-shared-requirements.md §3, collapsed onto one device. Every component still exists; several change form, and the graph should say so explicitly.

```
User (Product tab, native or React Native shell)
  → Device authentication (biometric / passcode)    ← replaces OIDC
  → Local authorization (which document sets)       ← role baked into provisioning
  → Query analysis
  → On-device retrieval (SQLite + vector index)
  → Skills: unit conversion, part lookup, checklists – all local
  → llama.cpp
    • GGUF Q4_K_M quantized weights
    • KV cache (RAM-bounded – surface it)
    • Metal / NNAPI / Vulkan offload
  → Guardrails + confidence gate
  → Local structured logs (stateless between sessions)
  → Local append-only audit log, hash-chained
  → Local metrics buffer
    ... opportunistic sync when connectivity returns ...
  → [Backend, when reachable] Prometheus → Grafana
  → Model registry → staged OTA rollout rings    ← replaces Kubernetes

    → Release rings (signed GGUF, staged OTA)
```

== RLHF / IMPROVEMENT LOOP ==

On-device rating + correction, written to a LOCAL queue
 → redacted on device, before the queue
 → upload is opt-in, explicit, per item – the queue is visible to the user, who can delete any of it

- Preference dataset (prompt, chosen, rejected)
- DPO fine-tune (LoRA, operator-run, never automatic)
- Promotion gate: `lm_eval` + on-device latency/memory harness

==> back to the shipped GGUF

Substitutions to state explicitly on the nodes (see shared requirements §4):

- **Kubernetes** □ **model registry with staged OTA rollout.** A phone does not run Kubernetes. The requirement is a controlled path from staging to production with rollback: signed model artifacts, versioned, released to a canary ring, then 10%, then general — with the ability to halt and roll back a bad model.
- **Server monitoring** □ **local buffer with opportunistic sync.** Metrics and audit events are written locally and uploaded when a network appears. They must survive app restarts and be capped so a device offline for weeks does not fill its storage.
- **Continuous batching** □ **not applicable.** Single user, one request at a time. Say so on the node rather than omitting it. The on-device analogue worth demonstrating is mmap loading and KV cache growth.

5. Frontend

React Native (or native shell with a React web view) so the three-tab structure is consistent with the rest of the pack.

Product tab — the assistant. Answers with citations into the source manual, page and section. Offline indicator prominently displayed. A **live resource readout**: tokens/sec, RAM in use, KV cache occupancy, battery drain.

That readout is not developer garnish — on mobile it is the product’s credibility. A client’s first question is “will this destroy the battery and fill the phone?”, and showing the answer live is the strongest possible response.

Configuration: document sets, model variant (Q4/Q5/Q8) with the trade-off table visible, context length, confidence threshold.

Architecture tab — 3D graph. Logos: llama.cpp, GGUF, SQLite, Apple Metal, Android, Prometheus, Grafana. Strongest copy needed on **GGUF quantization, mmap loading, KV cache, on-device retrieval and OTA rollout rings**.

Note: a 3D force graph on a phone needs a reduced node count and a lower frame rate cap, or it will drain battery visibly during the demo. Consider rendering the Architecture tab full-fidelity on tablet and a simplified layout on phone.

Monitoring tab — see below.

The Architecture tab must show the improvement loop. It is a stage card like any other, and it is the only one whose edges flow *back* upstream — that is the visual point. Give it the green treatment and its own tour stops; see §6 for the node inventory and `00-shared-requirements.md` §2 for the graph mechanics (expandable pipeline, click-to-isolate, draggable nodes, canvas-drawn icon glyphs, constant-size labels, HTML detail panel).

The Monitoring tab has five sections, not four — Quality, Performance, Security, Audit and **Improvement**. The last one is never seeded with demo data.

6. Architecture graph: node inventory

These are the stage cards for this project’s Architecture tab, in pipeline order. Every one is a node; every child is a node inside its parent’s card. Build them against the `ArchNode` shape in `00-shared-requirements.md` §8a, with all four rationale fields and a “Say this” line on each.

#	Stage card	Colour role	Expands into
0	User	neutral	— (asks, offline, on a plane)
1	Mobile App	teal	Chat UI, streaming renderer, model download manager, storage budget
2	Local Trust Boundary	amber	On-device authentication (biometric unlock), per-app sandbox, no network path — say this loudly, it is the product
3	On-device Context	violet	Local document index, embedding lookup, conversation history, prompt assembly
4	llama.cpp Runtime	orange	Qwen2.5-3B-Instruct (GGUF Q4_K_M) , mmap weight loading, Metal / NNAPI offload, KV cache, thread and batch tuning
5	Output Guardrails	violet	Confidence gating, refusal on unknown, citation into local documents
6	Local Store	blue	SQLite, conversation history, encrypted at rest with the platform keystore
7	Telemetry & Audit	magenta	Opt-in , aggregated, on-device audit log, crash reporting, no content leaves the device
8	Improvement Loop	green	On-device rating and correction, local-only queue , opt-in upload, preference dataset, DPO fine-tune, promotion gate ☐ <code>lm_eval</code> , on-device latency harness
9	Release Rings	grey	Model registry, signed GGUF artifacts, staged OTA rollout rings, rollback

Two cards carry this project’s argument and neither is the model. **Local Trust Boundary** is the reason a buyer chooses this over an API at all — nothing leaves the device — and **Release Rings** is the honest answer to “how do you ship an update to something with no server”. A phone does not run Kubernetes; the requirement is a controlled path with a rollback, not the specific tool.

7. The improvement loop for this project

00-shared-requirements.md §5a applies in full. What is specific here:

This is the hardest feedback loop in the pack, and being honest about why is worth more than pretending it is easy.

The device holds the signal and the network is the problem. A user who is offline by design cannot stream judgements to a server, and a product whose selling point is “nothing leaves the device” cannot quietly upload the conversations people corrected.

Design it as:

- **Collect locally, always.** Rating and correction are written to an on-device queue. This works offline and costs nothing.
- **Upload is opt-in, explicit, and per-item.** Not a settings toggle buried in a menu — a visible “send this correction to improve the assistant” action on the item itself. A blanket consent for a privacy-first product is a contradiction.
- **Redact on device, before the queue.** Not on the server. The point is that the raw text never leaves.
- **Show the queue.** Let the user see exactly what would be sent, and delete any of it. This is also the most persuasive privacy demo you can film.

Expect a low upload rate and say so in the README. A small volume of genuinely consented corrections is worth more than a large volume of quietly harvested ones, and it is the only version of this that survives a privacy review.

7a. LoRA adapters: where they earn their place here

Test this: the adapter is the update.

The hard problem in shipping an on-device model is not inference, it is distribution. A 4 GB GGUF re-download over mobile data is a thing users refuse, and it is the reason on-device assistants ship once and never improve.

A LoRA adapter is tens of megabytes. That changes what an update *is*:

Update type	Download	Realistic adoption
Full quantized model	~2–4 GB	poor — Wi-Fi only, deferred, often never
LoRA adapter	~20–80 MB	routine, like any app update

llama.cpp applies GGUF LoRA adapters at load time (or you can merge one into the base for a single artifact). Both paths are worth building and comparing:

- **Applied at load** — the base model stays on disk once, and each update ships only the adapter. Best distribution story, small runtime cost.
- **Merged ahead of time** — one artifact, no runtime overhead, but every update is a full re-download. Best inference story, worst distribution story.

Measure both and put the numbers in the README: load time, tokens/sec and peak RSS with an adapter applied versus merged versus base. On the oldest device in the support matrix. If applying at load costs meaningful memory, that decides it — memory is the hard ceiling on a phone, and exceeding it does not make the app slow, it makes the OS kill it.

Also test: per-vertical adapters. A field assistant for electricians and one for plumbers can be the same 3B base with different adapters. That is a product strategy that costs one download plus a small file per vertical, and it is impossible with a monolithic model.

8. The release gate for this project

00-shared-requirements.md §5b applies in full: two axes, either one able to block the release on its own.

Dimension	Question	Tool	Thresholds that matter here
Quality	Is it still right?	lm_eval on the desktop build of the same GGUF + held-out on-device suite	Run the gate against the exact quantized GGUF that ships , not the fp16 source. Q4_K_M is where the damage happens
Performance	Is it still fast enough on the worst phone you support?	On-device latency harness: p95 time-to-first-token, tokens/sec, peak RSS , thermal throttling onset, energy per reply	Peak memory is the hard ceiling. Exceeding it does not make the app slow, it makes the OS kill it — an absolute bound, not a relative one

GuideLLM targets a server; substitute an on-device harness and say so. Measure on the **oldest device in the support matrix**, sustained for several minutes — a benchmark that stops before the phone gets hot measures a state the user never experiences.

9. Monitoring and KPIs

Section	Metrics
Quality	Answer accuracy against the domain suite, citation rate, decline rate, user thumbs-down rate
Performance	Tokens/sec, TTFT, model load time, peak RAM , KV cache size vs context, battery drain per query , thermal throttling events
Security	Failed device authentications, attempts to access unauthorised document sets, integrity check failures on model artifacts
Audit	Local hash-chained log: every query, which model version, which document set, sync status
Improvement (RLHF)	Local judgements captured, opt-in upload rate (expect it to be low — say so), challenger win rate, pairs awaiting the next training run

The Improvement section is never seeded with demo data. Every other section falls back to synthetic numbers when the metrics backend is absent and says so with a badge. Feedback is a claim about what real people judged; a plausible approval rate nobody gave is the one number here that cannot be corrected by waiting for real traffic. An empty loop renders as empty.

Battery and thermal are first-class metrics here, not curiosities. A model that is fast for ten minutes and then thermally throttles to unusability has failed, and this only shows up under sustained load on real hardware. Measure it across a realistic session, not a single query.

Report per device tier. A flagship phone and a three-year-old mid-range device are different products, and the client needs both numbers.

10. Security, audit and compliance

The offline architecture is the security story — but it must be built, not claimed:

- **Model artifacts are signed and verified before load.** An unsigned model file is arbitrary code execution risk on a device you do not control.
 - **Documents encrypted at rest** with a key in the platform keystore (Keychain / Android Keystore), released by biometric authentication.
 - **The audit log is local, append-only and hash-chained**, and it syncs when connectivity allows. Tamper-evidence matters more here, not less, because the device is physically in an untrusted environment.
 - **Remote wipe** for lost devices — a real requirement for regulated deployments and one clients ask about immediately.
 - **GDPR:** on-device processing is a strong data-minimisation and lawful-basis story. State it accurately: data does not leave the device, so there is no transfer and no third-party processor for the inference step.
-

11. Deployment

- iOS via TestFlight, Android via Play internal → closed → open testing tracks. That progression *is* staging → production; map it explicitly in the graph.
 - **Models ship separately from the app binary.** A 4.4 GB model inside an app bundle is a hostile download and makes every code fix a full re-download. Deliver models as signed, versioned artifacts fetched on first run and updated independently.
 - Staged rollout rings with halt-and-rollback. A bad quantization reaching every device at once, on hardware you cannot reach, is the worst failure mode this project has.
-

12. Deliverables

- Working mobile app, fully functional in airplane mode
- GGUF quantization pipeline producing three variants with a measured comparison table
- Quality gate that runs **on-device**
- All three tabs
- Local audit log with hash chain and sync
- Model registry with signed artifacts and staged rollout
- Measured performance across at least two device tiers

Acceptance: 1. Full demo completes in airplane mode with no network calls (prove it with a packet capture, not an assertion) 2. Sustained session on a mid-range device without thermal collapse, with the curve recorded 3. Audit log hash chain verifies, and a tampered entry is detected 4. A model artifact with a bad signature is refused 5. Architecture tab and guided tour run offline

13. Out of scope

Multi-user on one device. Real-time sync between devices. Fine-tuning on-device. Voice input — that is Project 01. Corpora larger than device storage allows, which would need a fundamentally different retrieval design.